

COMP2048: Theory of Computing

Dr. Josh Arnold
Week 7

16 April 2026

Where we've been

Over the past two weeks:

- ▶ **Turing machines:** formal 7-tuple, tape + head, the Universal TM
- ▶ **T -recognizable vs. T -decidable**
Quick check: what's the difference? (Which one always halts?)
- ▶ A_{TM} and $HALT_{TM}$ are **undecidable** (via diagonal arguments and reductions)
- ▶ **Church-Turing Thesis:** TMs capture “anything you'd call an algorithm”

Today: a model of computation that looks *nothing* like a Turing machine. Yet it computes everything a Turing machine can.

A different shape of computation

Every model we've met so far has been:

- ▶ **Sequential**: one step at a time
- ▶ **Centralised**: a single state, a single head, a single locus of control

A **cellular automaton** is none of those:

- ▶ A grid of cells, each holding a value from a small alphabet
- ▶ No central controller; each cell only “sees” its immediate *neighbours*
- ▶ Every cell updates **in parallel**, in lockstep, by the same local rule

Simple local rules $\implies$ rich global behaviour.

That sentence is the whole lecture.

Today's plan

First half (before the break), *formalisms*:

- ▶ What a cellular automaton *is*
- ▶ Wolfram's enumeration of the 1D rules
- ▶ Four species of behaviour, and one rule we can prove is *as hard as the halting problem*

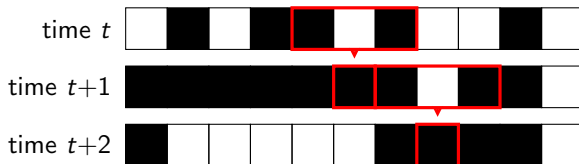
Second half (after the break), *exploration*:

- ▶ Conway's Game of Life
- ▶ Self-reproducing patterns
- ▶ Emergence and the “edge of chaos”

Remember the closing slide from two weeks ago, “Accidentally Turing-Complete”? One of the names on that list was **Rule 110**. By the break, we'll see exactly why it belongs there.

What is a cellular automaton? (the picture)

- ▶ A **grid of cells**, each holding one of finitely many values.
- ▶ Each cell “sees” a fixed set of nearby cells: its **neighbourhood**.
- ▶ A single **local rule** computes each cell's next value from what it currently sees.
- ▶ All cells apply the rule **simultaneously**, in discrete time steps.



Cellular automaton (formally)

A **cellular automaton** is a tuple (L, Σ, N, δ) :

- ▶ L , the **cellular space**: a regular lattice, e.g. $\mathbb{Z}$, $\mathbb{Z}^2$, or a finite grid.
- ▶ Σ , a finite **state set**: the values a cell can take.
- ▶ N , the **neighbourhood**: a finite set of offsets, fixing which cells influence a given cell's next value.
- ▶ $\delta : \Sigma^{|N|} \rightarrow \Sigma$, the **local rule**: given the values in a neighbourhood, returns the cell's next value.

A *configuration* $c : L \rightarrow \Sigma$ assigns a state to every cell.

The **global rule** Δ maps configurations to configurations by applying δ at every cell **in parallel**.

“In parallel” is literal. All new values computed from the old configuration, then updated at once.

Finite grids: what about the edges?

On an infinite lattice, Δ is unambiguous. On a finite grid, a cell near the edge is missing some neighbours. Three standard conventions:

- ▶ **Zero-padded** (fixed): treat cells outside the grid as a fixed background value, usually 0.
- ▶ **Cyclic** (wrap-around): in 1D, the line becomes a ring; in 2D, a torus.
- ▶ **Reflecting**: mirror the grid at each boundary.

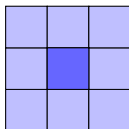
Boundary choice is part of the specification, not a detail. The same rule on the same initial configuration can produce qualitatively different long-run behaviour under different boundary conditions. We will come back to this.

Choosing the neighbourhood

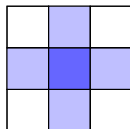
The neighbourhood N is a design choice. Two families we will meet today:

In 1D: the *radius- r* neighbourhood is the cell itself plus r cells on each side, so $2r + 1$ cells in total.

In 2D: two standard shapes.



Moore (9 cells)



von Neumann (5 cells)

Preview: Conway's Game of Life, covered after the break, uses Moore.

Elementary cellular automata

For now, we work with the simplest nontrivial setting.

- ▶ Cellular space: a 1D line of cells, $L = \mathbb{Z}$ (or a finite ring).
- ▶ State set: $\Sigma = \{0, 1\}$.
- ▶ Neighbourhood: radius 1, offsets $\{-1, 0, +1\}$. Three cells.

A local rule $\delta : \{0, 1\}^3 \rightarrow \{0, 1\}$ has $2^3 = 8$ possible inputs, and assigns a bit to each. So the number of possible rules is

$$2^{2^3} = 2^8 = 256.$$

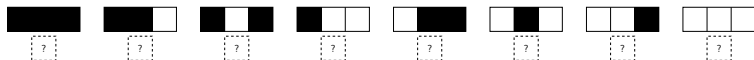
All of them. Few enough to enumerate.

That is what Wolfram did in the 1980s, and it is where we go next.

Encoding a rule: a lookup table of 8 bits

An ECA rule is a function $\delta : \{0,1\}^3 \rightarrow \{0,1\}$.

It is fully specified by its output on each of the 8 possible inputs.

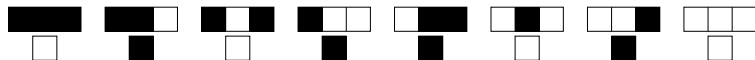


Each “?” is a bit. **8 bits** of data total, so $2^8 = 256$ rules.

Wolfram's numbering: read the 8 output bits left to right as a binary number. That number $\in \{0, \dots, 255\}$ names the rule.

Worked example: Rule 90

Fill in the outputs that define Rule 90:



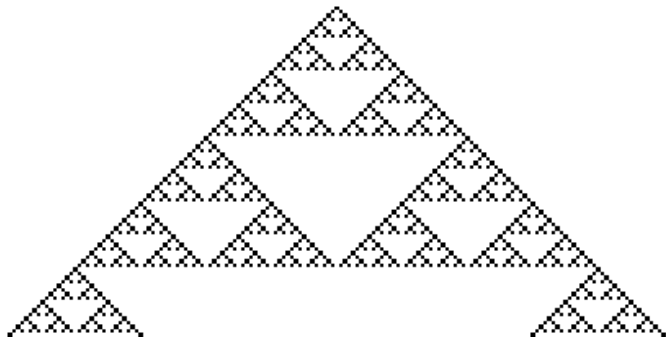
Outputs, left to right: 01011010.

$$01011010_2 = 64 + 16 + 8 + 2 = \mathbf{90}.$$

Look again at the table. The output is 1 exactly when the left and right neighbours differ: Rule 90 is $\text{next} = \text{left} \oplus \text{right}$.

Rule 90 from a single seed

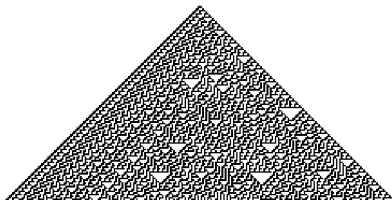
Run Rule 90 on a row of zeros with a single 1 in the middle. Time flows downward.



That is the **Sierpinski triangle**: a famous fractal, here produced by the simplest conceivable XOR rule on a row of bits.

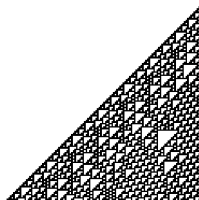
Two more rules: 30 and 110

Rule 30



A single seed, zero elsewhere. Looks like TV static. Wolfram used the centre column of Rule 30 as *Mathematica's* default random number generator for decades.

Rule 110



Same initial condition, different rule. Persistent structures drift across a textured background. We will come back to this one.

Same setup, same seed, three very different species of behaviour.

Any of the 256 rules, in three lines

The Wolfram number n is the lookup table:

```
# n is the rule number, in [0, 255]
rule_bits = [(n >> i) & 1 for i in range(8)]

# at each cell, given its neighbourhood:
idx      = (left << 2) | (centre << 1) | right
next_cell = rule_bits[idx]
```

Change n , change the rule. No other code moves. We'll run a lot of rules this way in the next section.

A different question: what do rules *typically* do?

So far we've watched three rules from a **single seed**. Clean, but atypical: a single 1 on a zero background is a very specific input.

Different question:

- ▶ Pick a rule $n \in \{0, \dots, 255\}$.
- ▶ Start from a **random row of bits** on a ring.
- ▶ Run it for a few hundred steps. What does the space-time diagram look like?

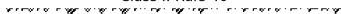
Do this for every rule and sort by what you see.

That is an **empirical taxonomy** of the 256 ECAs. Wolfram did it in the early 80s.

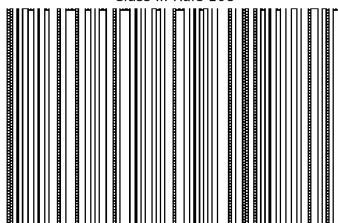
Four classes of behaviour

One representative rule per class, same random initial row:

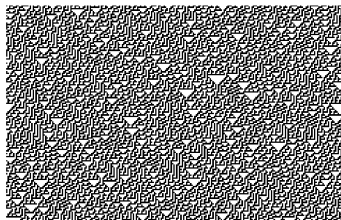
Class I: Rule 40



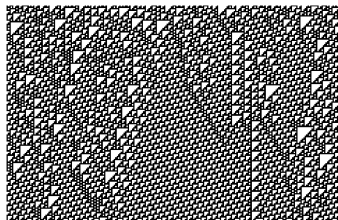
Class II: Rule 108



Class III: Rule 30



Class IV: Rule 110



Four qualitatively different long-run behaviours. And (empirically) every ECA falls into one of these four buckets.

The four classes

Class I <i>homogeneous</i>	Dies to a fixed state (usually all 0s). Every initial row is forgotten.	Rules 0, 32, 40, 160
Class II <i>periodic</i>	Settles into stable structures or short repeating cycles. Stripes, blocks.	Rules 4, 56, 108
Class III <i>chaotic</i>	No persistent structure; looks like noise forever. Passes statistical tests for randomness.	Rules 30, 90, 150
Class IV <i>complex</i>	Localised structures that move, col- lide, transform, against a textured background.	Rules 54, 110

Class I and II are boring. Class III is noisy. Class IV is strange.

Class IV is rare, and Class IV is where it's interesting

Out of 256 elementary rules, only a handful are Class IV. The overwhelming majority of ECAs are I, II, or III.

Class IV sits right between:

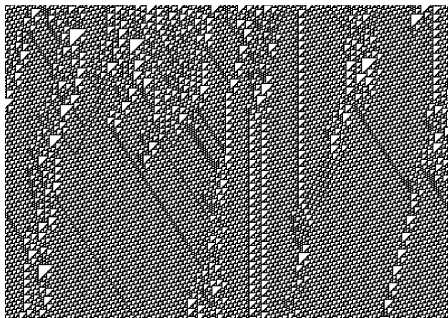
- ▶ **Class II** (order): structure that freezes, loses all memory of the input.
- ▶ **Class III** (chaos): structure that scrambles, any signal drowns in noise.

Chris Langton (1990) named this in-between regime the **edge of chaos**. We'll return to it after the break.

And one more thing about Class IV.

Class IV is exactly where **computation** can live. Rule 110, on the list above, is Turing complete. That is where we go next.

Rule 110, up close



What we see:

- ▶ A textured diagonal **ether** (the background pattern).
- ▶ Persistent localised structures drifting across: **particles** (or **gliders**).
- ▶ Different particle species travel at different speeds. They **collide** and transform into new particles, or annihilate.

Rule 110 is Turing complete

Theorem (Cook, 1998 / 2004). Given any Turing machine M and input w , there is an initial Rule 110 configuration whose evolution *simulates* M on w .

Sketch (very): Cook showed

Rule 110 simulates cyclic tag systems, which simulate Turing machines.

Particles are signals; collisions implement logic; the right initial particle arrangement encodes any computation.

The actual construction is 40+ pages. We will not reproduce it. We **use it as a black box**.

*A 1D binary radius-1 rule is Turing complete.
That is astonishing.*

A seemingly innocent question

Given a Rule 110 initial configuration c and a cell position i : *does cell i ever become 1?*

Formally:

$\text{CELL} = \{ \langle c, i \rangle : \text{cell } i \text{ equals 1 at some time } t \geq 0 \text{ under Rule 110 from } c \}$.

Theorem. CELL is undecidable.

In words: there is no algorithm that, given $\langle c, i \rangle$, always terminates with the right yes/no answer.

We prove it by **reduction from** HALT_{TM} : the same move from last week's lecture 6, just with a new target.

Reduction $\text{HALT}_{TM} \leq_m \text{CELL}$

Suppose for contradiction some TM D decides CELL. Build a decider for HALT_{TM} :

On input $\langle M, w \rangle$:

1. Use Cook's construction to build a Rule 110 initial configuration $c_{M,w}$ and a distinguished cell position i^* , such that

cell i^* becomes 1 $\iff M$ halts on w .

2. Run D on $\langle c_{M,w}, i^* \rangle$.
3. **Accept** iff D accepts.

If D decides CELL, this procedure decides HALT_{TM} .

But HALT_{TM} is undecidable. $\Rightarrow$ No such D exists. ■

A question about a row of bits updated by one XOR-like rule, that no computer will ever answer.

Conway's Game of Life

A 2D cellular automaton. Binary states, Moore neighbourhood (the 8 surrounding cells, introduced earlier).

Rule B3/S23. For each cell:

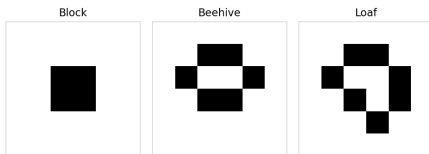
- ▶ **Birth:** a dead cell with *exactly* 3 live neighbours comes alive.
- ▶ **Survival:** a live cell with 2 or 3 live neighbours stays alive.
- ▶ **Death:** every other cell dies (“loneliness” with 0 or 1, “overcrowding” with 4+).

Invented by John Conway in 1970; popularised by Martin Gardner in *Scientific American* that October. Still the most famous CA.

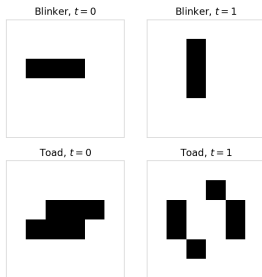
Three lines, two states. Deceptively simple.

A pattern zoo, part 1: still lifes and oscillators

Still lifes hold their shape forever:



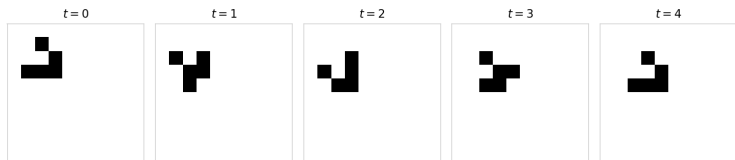
Oscillators cycle through a fixed sequence of shapes:



Both shown have period 2. Larger oscillators exist with periods 3, 15, even millions.

A pattern zoo, part 2: the glider

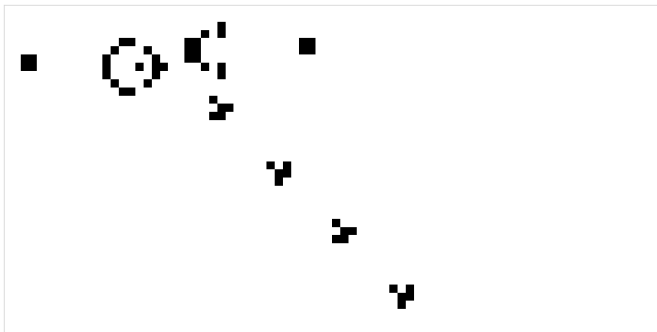
Spaceships are patterns that translate across the grid. The smallest is the **glider** (Richard Guy, 1970):



Period 4; after one cycle the glider is shifted by one cell diagonally.
Speed: $c/4$ diagonal.

The first pattern anyone found that *moves*. Everything universal about Life, we'll see, is built on top of gliders.

Gosper's glider gun



Emits one glider every 30 steps, forever.

Conway put up a \$50 prize for any pattern of **unbounded growth**. Bill Gosper's team at MIT won it in November 1970 with this gun. Glider gun = population grows without bound.

The gun is also the first step of the universality proof: a way to *manufacture* the signals the circuit needs.

Life is Turing complete

Gliders are the particles of Life: localised, moving, interacting.

Logic gates can be built from glider collisions:

- ▶ Two streams of gliders meeting at the right angle can **annihilate** (AND-NOT).
- ▶ “Eater” patterns absorb stray gliders without getting damaged (NOT).
- ▶ Guns supply glider streams on demand (signal source).

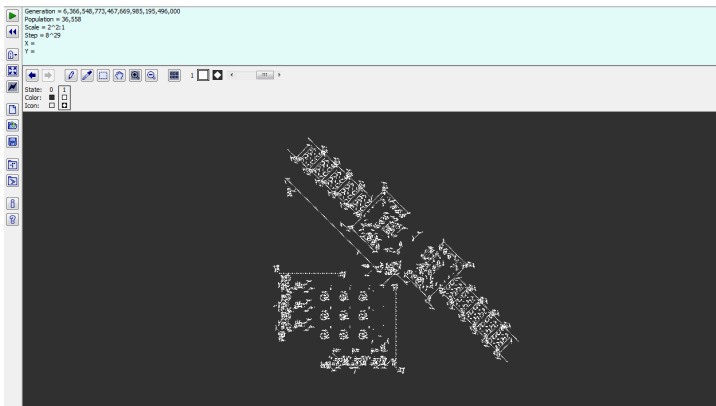
Wire enough of these together, with the right timing, and you get a full Turing machine. Paul Rendell (2000) built a compact one; later constructions have simulated specific Turing machines running Life inside Life.

Class IV in 2D, same story as Rule 110 in 1D: simple local rule, universal computation.

Programmable computer with display, built in Life:

<https://www.youtube.com/watch?v=WfuhbI8HE7s>

Rendell's Turing machine in Life



A running Turing machine built entirely from gliders, guns, and eaters.
Generation count visible top-left: this snapshot is at step 6.3×10^{27} , computed by Golly in under 30 seconds using Hashlife.

Image: Wikimedia Commons, CC BY-SA. Full construction and patterns:

<http://rendell-attic.org/gol/tm.htm>

Can self-reproduction be formalised?

Living things reproduce. Can a *machine*?

John von Neumann (late 1940s, early 1950s) took the question seriously. He wanted a mathematical model in which a machine could build a copy of itself from raw materials on a uniform “workspace.”

His answer, around 1952: a 2D cellular automaton with

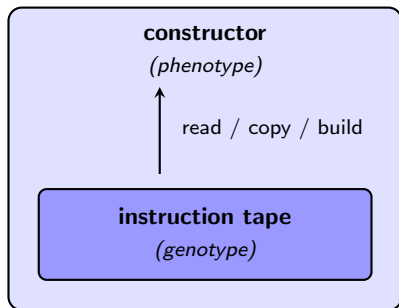
- ▶ 29 cell states,
- ▶ von Neumann neighbourhood (introduced earlier),
- ▶ and a single configuration, hundreds of thousands of cells, that copies itself.

Too large to simulate on the hardware of the time. Nobili and Pesavento finally ran it in 1995, forty years after von Neumann's death.

Genotype and phenotype

von Neumann's self-reproducer has two pieces:

- ▶ An **instruction tape** (the *genotype*): passive data describing how to build the machine.
- ▶ A **universal constructor** (the *phenotype*): active machinery that reads the tape and assembles a copy.



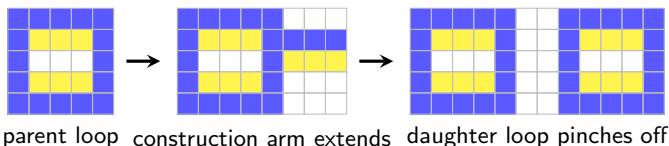
Reproduction: *copy* the tape, then *build* a new constructor from it.

Historical beat. Watson and Crick published the double helix in 1953. von Neumann nailed this split *before* anyone knew what the molecule looked like. Tape $\leftrightarrow$ DNA; constructor $\leftrightarrow$ ribosome. Prescient.

Simpler loops

von Neumann's construction was too large to actually run. A line of simplifications:

- ▶ **E. F. Codd (1968)**: same idea, collapsed to **8 states**. Still big; still rarely simulated.
- ▶ **Christopher Langton (1984)**: dropped *universality*, kept the self-copying core. An **86-cell loop**, 8 states, von Neumann neighbourhood. Small enough to sit on a slide.



A tape of signals circulates inside the loop. The arm extends as signals reach the corner; when the arm closes back on itself, it pinches off a daughter carrying a copy of the tape. The visual reads as biological: cells dividing.

Video: <https://www.youtube.com/watch?v=xsLX390SJM4&t=546s>

Self-reproduction inside Life

Can a pattern in Conway's Game of Life reproduce itself?

Gemini (Andrew Wade, 2010): the first fully self-constructing replicator in Life.

- ▶ ~850,000 live cells.
- ▶ Reproduces once every ~33.7 million generations.
- ▶ Uses a construction arm built from gliders, following a recipe tape: the same genotype/phenotype split von Neumann described.

OEOP metacell (2010s): a Life pattern that simulates *any* cellular automaton, including Life itself. Life running inside Life.

Video: <https://www.youtube.com/watch?v=xP5-iIeKXE8>

Same question, answered four different ways, over sixty years.

The through-line

Every example today has had the same shape.

- ▶ **Rule 90** (XOR in one line) $\Rightarrow$ the Sierpinski triangle.
- ▶ **Rule 110** (an 8-bit table) $\Rightarrow$ a universal computer.
- ▶ **Game of Life** (rule B3/S23) $\Rightarrow$ gliders, guns, logic gates.
- ▶ **Langton's loops** (8 states, 86 cells) $\Rightarrow$ self-reproduction.

Each local rule fits on a sticky note. Each global behaviour fills a textbook.

That is **emergence**: structure that is not present in any single cell's rule, yet arises from running the rule everywhere in parallel.

One last question: can we predict, from the rule alone, which regime a CA lands in?

Langton's λ : one number per rule

For a binary CA, define

$$\lambda = \frac{\# \text{ neighbourhood patterns whose output is 1}}{\# \text{ possible neighbourhood patterns}}.$$

For an ECA (8-entry lookup table), $\lambda \in \{0, \frac{1}{8}, \frac{2}{8}, \dots, 1\}$.

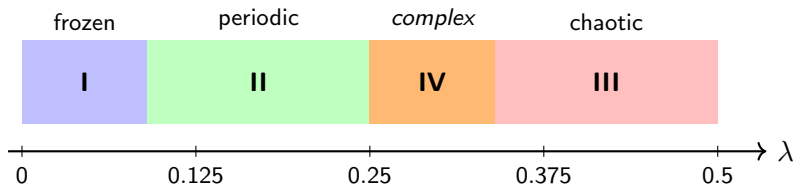
Concretely, λ is the Hamming weight of the Wolfram number, divided by 8.

Examples.

- ▶ Rule $0 = 00000000_2$: $\lambda = 0$. Everything dies.
- ▶ Rule $4 = 00000100_2$: $\lambda = 1/8$.
- ▶ Rule $30 = 00011110_2$: $\lambda = 4/8$.
- ▶ Rule $110 = 01101110_2$: $\lambda = 5/8$.
- ▶ Rule $255 = 11111111_2$: $\lambda = 1$. Everything becomes 1.

By $0 \leftrightarrow 1$ symmetry, λ and $1 - \lambda$ give mirror-image dynamics. So the informative range is $\lambda \in [0, 0.5]$.

Sweep λ : the phase picture



As λ grows from 0 to 0.5:

- ▶ small λ : frozen dynamics (Class I).
- ▶ modest λ : periodic structures (Class II).
- ▶ a narrow band near the transition: complex (Class IV).
- ▶ large λ : chaotic noise (Class III).

Class IV does not sit in the middle of the axis. It sits on the phase boundary between order and chaos.

Computation at the edge of chaos

Langton (1990), “Computation at the edge of chaos.”

- ▶ **Order** (I, II) *stores* information but never moves it. No computation.
- ▶ **Chaos** (III) *moves* information everywhere but scrambles it. No computation.
- ▶ **Edge** (IV) does both: structures hold their shape *and* they interact.

Consistent with everything we saw today:

- ▶ **Rule 110** sits at the edge. Turing complete.
- ▶ **Conway's Life** sits at the edge. Turing complete.
- ▶ Every other CA proved universal clusters here too.

A framework, not a theorem. Still, it unifies every visual we watched today.

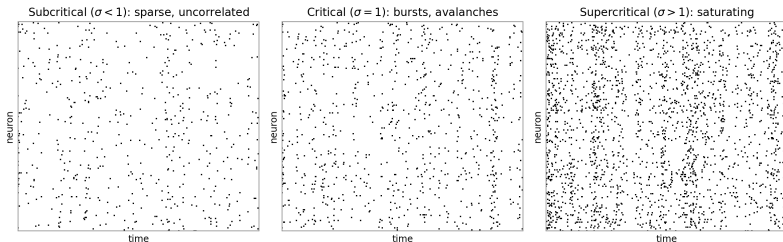
Rich computation does not live in boring rules, and does not live in chaotic rules.

It lives on the boundary between them.

The same story in real brains

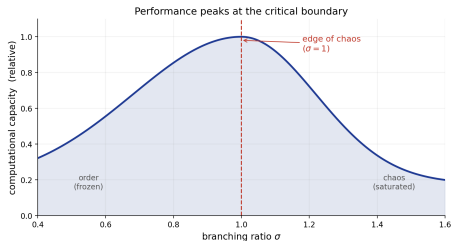
A **spiking neural network** (SNN): neurons fire discrete spikes when their membrane potential crosses a threshold. Local rules, parallel updates, discrete time (the same four ingredients as a CA).

Same three regimes, different substrate



Beggs & Plenz (2003) recorded spontaneous activity in cortical slices and found *neuronal avalanches*: burst sizes following a power law, the canonical signature of criticality. The middle regime is not just a CA curiosity; it is what cortex actually does.

Why the edge? Computational capacity



Kinouchi & Copelli

(2006): dynamic range of an excitable network is mathematically maximised at criticality.

Maass et al. (2002):

liquid state machines, SNNs tuned near the chaotic transition, show peak memory and separation capacity. Cites Langton directly.

Chialvo (2010): review of brain criticality, arguing cortex self-organises to this regime.

Many substrates, one sweet spot

Neuromorphic hardware: chips that run spiking dynamics directly in silicon.

- ▶ Intel **Loihi**, IBM **TrueNorth**, SpiNNaker (Manchester).
- ▶ Tuning operating regime toward criticality is an active design principle.

Langton's edge of chaos, seen across today's lecture and beyond:

- ▶ Elementary CAs (Rule 110): universal, at the edge.
- ▶ Conway's Life: universal, at the edge.
- ▶ **Cortex**: neuronal avalanches at the edge (Beggs & Plenz).
- ▶ SNN reservoirs: computational capacity peaks at the edge (Maass).
- ▶ Neuromorphic chips: tuned there by design.
- ▶ **Deep neural networks**: trainable only when initialised near the edge.

One framework, six substrates. Rule 110 is not an exotic curiosity; the phenomenon lives in your head (and inside every large model you have heard of).

Closing the loop

We opened with a promise: a model of computation that looks *nothing* like a Turing machine, yet computes everything a TM can.

The list of universal models you now know:

- ▶ Turing machines.
- ▶ λ -calculus, tag systems, register machines.
- ▶ **Cellular automata**: Rule 110, Conway's Game of Life.
- ▶ Spiking / recurrent networks (Siegelmann & Sontag, 1991).

Same computational power. Wildly different shape.

The Church-Turing thesis just acquired one more data point (and the edge-of-chaos phenomenon a physical home, in cortex).

Simple local rules $\implies$ rich global behaviour $\implies$ full computational power.

The whole lecture, in one sentence.

Where to go next

Course materials.

- ▶ Notebooks on the course site: ECA explorer, Game of Life, Langton's ant.
- ▶ **Assignment 2, Q5** draws directly on today's content.

Going deeper.

- ▶ Wolfram, *A New Kind of Science* (2002). Free online; a compendium.
- ▶ Cook, "Universality in Elementary Cellular Automata," *Complex Systems* (2004). The full Rule 110 proof.
- ▶ Berlekamp, Conway, Guy, *Winning Ways*, vol. 2, ch. 25 (1982). The classic Life chapter.
- ▶ Schiff, *Cellular Automata: A Discrete View of the World* (2008).
- ▶ **Edge of chaos in cortex:** Beggs & Plenz (2003, *J. Neurosci.*); Chialvo (2010, *Nat. Physics*); Maass et al. (2002, *Neural Comp.*).

Play around.

- ▶ conwaylife.com: pattern catalogue, in-browser simulator.

Thanks!

Questions?

Next week: handed back to Kasra.